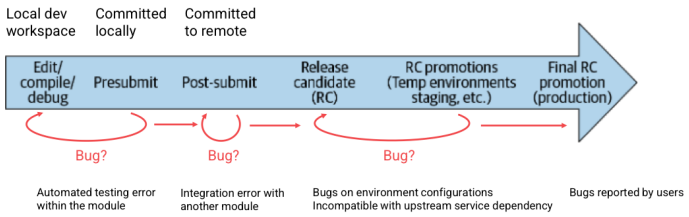
Life of a code change



To minimize the cost of bugs, we need a feedback loop whenever we integrate a code change

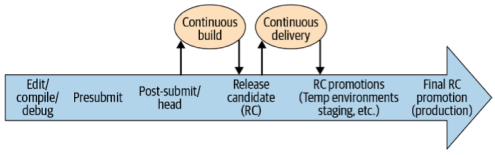
尽早将bug反馈

Continuous Integration (CI)

- CI is a software development practice where members of a team integrate their work frequently
- Each integration is verified by an automated build (including test) to detect integration errors as quickly as possible

The fundamental goal of CI is to automatically catch problematic changes as early as possible.

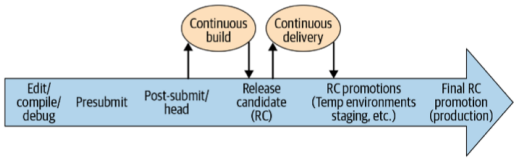
- CI, specifically, automates the build and release processes, with a Continuous Build and Continuous Delivery.
- Continuous testing is applied throughout the entire process (e.g., unit/integration/system test, etc.)



Continuous Build (CB)

- CB integrates the latest code changes at HEAD
- CB runs automated build
- CB runs automated tests
- If the code passes all tests, CB marks it passing

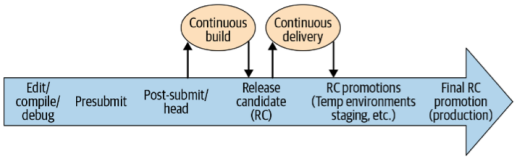
"Breaking the build" or "failing the build" includes breaking compilation or breaking tests



- Release candidate (RC): a cohesive, deployable unit created by an automated process, assembled of code, configuration, and other dependencies that have passed the continuous build.

Continuous Delivery (CD)

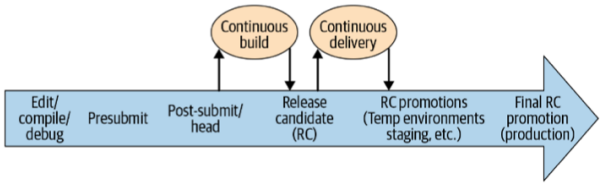
- CD: a continuous assembling of release candidates, followed by the promotion and testing of those candidates throughout a series of environments—sometimes reaching production and sometimes not.



Continuous Testing

- Continuous testing is applied throughout the CI process
- Key objective: determining **when** to test **what**

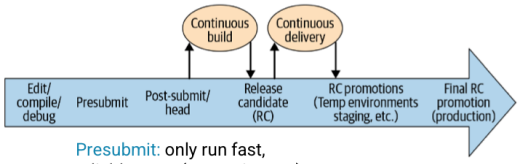
Why not just run all tests on presubmit?



Presubmit

Case study at Google

- Each team creates a **fast subset** of tests, often a project's unit tests, that can be run before a change is submitted and code reviewed (i.e., presubmit)
- Empirically, a change that passes the presubmit has a very high likelihood (95%+) of passing the rest of the tests

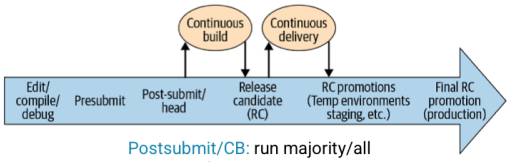


Presubmit: only run fast, reliable tests (e.g., unit tests)

Presubmit/CB

Case study at Google

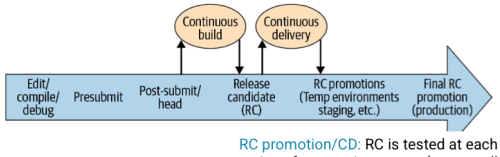
- After a change has been submitted, the Test Automation Platform (TAP) run all potentially affected tests, including larger and slower tests
- When a change causes a test to fail on TAP, engineers decide whether to fix it or roll back (TAP enables automatic rollback on high-confidence cases)



Postsubmit/CB: run majority/all automated tests

RC Promotion/CD

- Staging (类生产环境) is a test environment that exactly resembles a production environment
- It seeks to mirror an actual production environment as closely as possible and may connect to other production services and data, such as databases and servers.

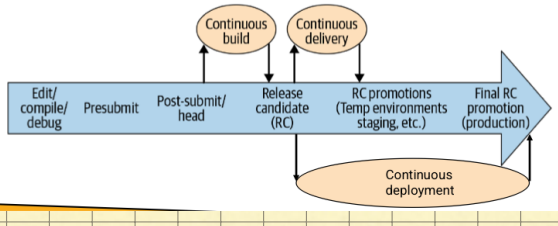


RC promotion/CD: RC is tested at each deployment on a series of test environments (e.g., sandbox, staging)

- The primary use of a staging environment is to test all the installation/configuration/migration scripts and procedures before they're applied to a production environment.
- Another important use of staging is performance testing, particularly load testing

Continuous Deployment

- CI/CD is (less often) referred to as continuous integration / continuous deployment
- Difference between continuous delivery and continuous deployment
- Continuous delivery automates deployment of a release to a staging or testing environment
  - Make sure that software changes are always in a releasable state (can be deployed at any time)
  - Decision of whether to deploy changes to production is left to the team (human intervention)
- Continuous deployment: automates deployment of a release all the way to the production environment, no human intervention



Risks of Continuous Deployment

- New production version has problems and needed to be rolled back to previous versions
- Service is temporarily down during upgrading or rollback
- ……

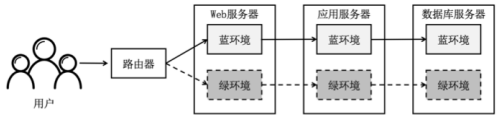
Deployment Strategies

Goal: minimize the impact of new deployment on end users

- Blue-Green Deployment (蓝绿部署)
- Canary/Greyscale Release (金丝雀发布/灰度发布)

Blue-Green Deployment

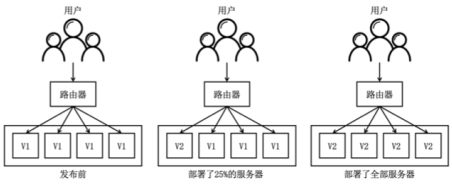
- A deployment strategy in which you create two separate, but identical environments.
- Blue environment is running the current/old application version
- Green environment is running the new application version.
- Once testing has been completed on the green environment, live application traffic is directed to the green environment.
- If error occurs, switch the traffic back to the blue environment



两个环境并行存在
蓝：当前/旧版本
绿：新版本

Canary Release (Greyscale Release)

- A deployment strategy to reduce the risk of introducing a new software version in production
- Slowly rolling out the change to a small subset of users (canary) before rolling it out to the entire infrastructure and making it available to everybody.
- Once error occurs, simply stop the traffic to the server with new version



新服务发布时，先将新服务发布给一部分用户，再逐步发布给其他用户。出错时立刻回滚。

Traditional Software Development

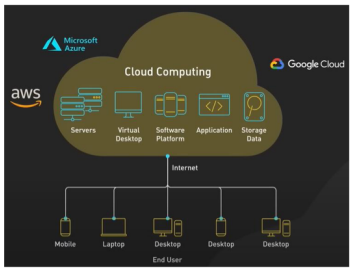
On-premise (本地部署): organizations need to purchase and maintain their own hardware and software in-house, including servers, storage devices, and networking equipment.



- Significant manual intervention
- Long release cycle
- Complex, time consuming, error-prone

Cloud Computing

Running applications on computing resources managed by cloud providers (e.g., aws), without having to purchase and manage hardware ourselves



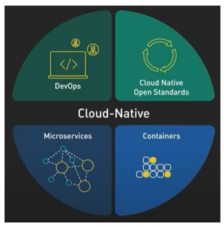
- Free the team from managing hardware infrastructure
- Fast to support new computing resources
- Scaling up is effortless
- Cost-effective: cloud providers offer pay-as-you-go pricing models

Cloud Native Applications

Cloud native apps are designed and built to exploit the scale, resiliency, and flexibility the cloud provides.

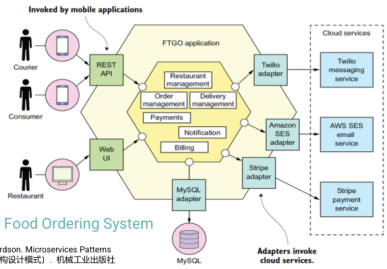
For an application to be considered as cloud-native, it should at least adopt these 4 approaches or technologies

- DevOps
- Microservices
- Containers
- Cloud-native Open Standards



Microservices

Cloud-native applications consist of multiple small, interdependent services called microservices.



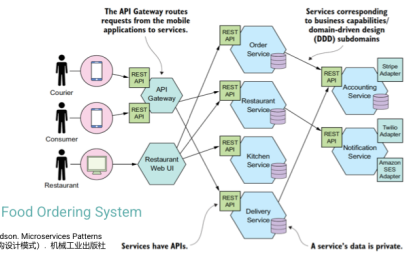
Online Food Ordering System

Chris Richardson. Microservices Patterns（微服务架构设计模式）. 机械工业出版社

Monolithic architecture

- The whole application is packaged into a single executable
- Less flexible for large team/code base
- Difficult to scale, wasting deployment resources

Cloud-native applications consist of multiple small, interdependent services called microservices.



Online Food Ordering System

Chris Richardson. Microservices Patterns（微服务架构设计模式）. 机械工业出版社

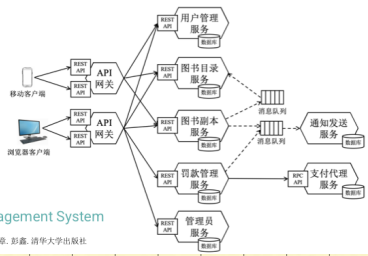
Microservice Architecture

- Microservices allow a large application to be separated into smaller independent parts, with each part having its own responsibility
- Each service can be developed, managed, and deployed independently.

Microservice Architecture

- Services communicate with each other by using well-defined APIs (e.g., REST)
- We could scale only the required microservice

Cloud-native applications consist of multiple small, interdependent services called microservices.



Library Management System

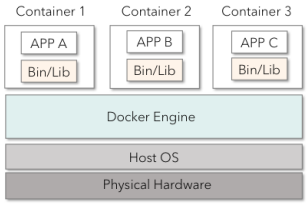
现代软件工程基础,第7章. 彭鑫. 清华大学出版社

Microservice Architecture

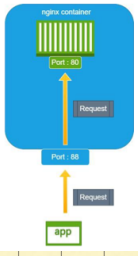
- Services communicate with each other by using well-defined APIs (e.g., REST)
- We could scale only the required microservice

Containers

A container is a lightweight and portable unit of software that packages an application and its dependencies together in a single environment.

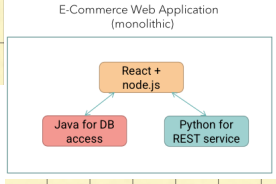


- A container consists of all the dependencies required to run an application, and isolates these dependencies from other containers on the same machine
- Developers can easily create, deploy, and run applications in a consistent and predictable manner across different servers or cloud platforms.



Container Communication

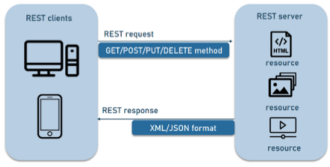
- Containers are assigned to unique IP addresses so that they could communicate with each other using standard network protocols (e.g., TCP/IP)
- Containers could expose services to the outside world by mapping container's port to the port on the host machine (port forwarding)



Communications

REST Services

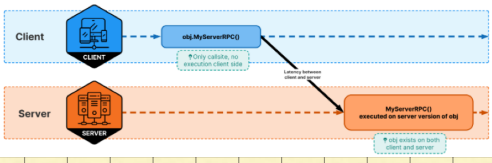
- RESTful systems use HTTP verbs to manipulate resources identified by URIs (Uniform Resource Identifiers).
- RESTful APIs are stateless and cacheable, which makes them highly scalable and easy to manage.
- Clients can manipulate resources through simple and standardized interfaces.



https://www.altexsoft.com/blog/rest-api-design/

RPC (Remote Procedure Call) Services

- RPC is a mechanism for invoking a function or method on a remote server, and receiving the results back.
- When calling an RPC, you call a method remotely on an object that can be anywhere in the world (i.e., RPC methods execute on a different machine)

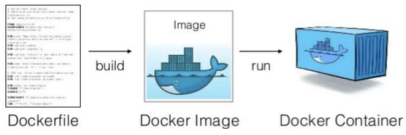


Docker

Docker is one of the most popular platforms for building, deploying, and managing containers

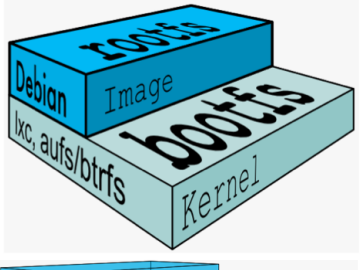
Docker image (镜像): a read-only template that acts as a set of instructions to create a container.

Dockerfile: a script that defines the instructions for building the image

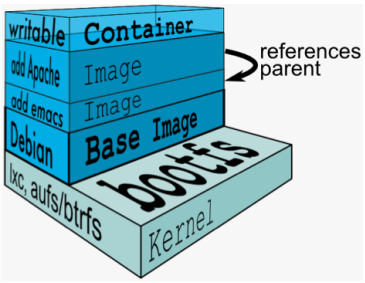


Docker container: a running instance of a Docker image

Docker Images



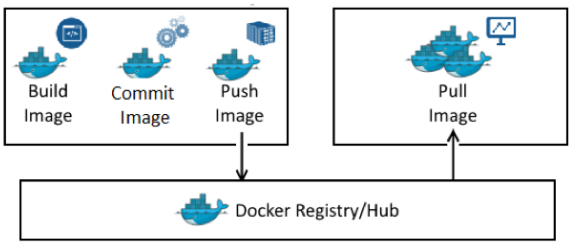
- Docker images consist of many layers. Each layer is built on top of another layer to form a series of intermediate images
- Kernel: Host machine's kernel
- Base image:
  - Different Linux distribution, e.g., Debian, Ubuntu
  - Various pre-installed software images, e.g., OpenJDK
  - Or a blank image with nothing included



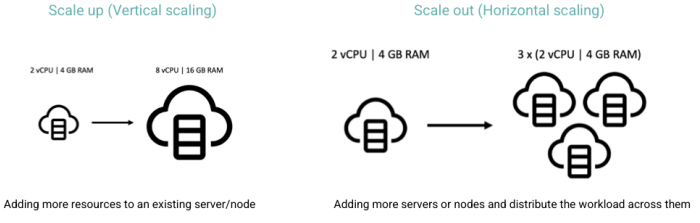
- Images (read-only) can be layered on top of one another, specified by Dockerfile
- Each time Docker launches a container from an image, it adds a thin writable layer, known as the container layer, which stores all changes to the container throughout its runtime.

- The top layer has read-write permissions, and all the remaining layers have read-only permissions
- Several containers may share access to the same underlying level of a Docker image, but write the changes locally and uniquely to each other

Docker image could be shared as we did for source code version control



Scale



增加资源体积、 增加资源数量.

Scale out – complexities

- Load Balancing: With more servers added, it's important to distribute incoming traffic across these servers to ensure that the load is evenly distributed
- Configuration: each new server or node needs to be configured properly to work with the rest of the infrastructure (e.g., network config, security config). As the number of servers grows, the complexity of configuring and managing them can increase.
- Monitoring/Troubleshooting: With more servers in the infrastructure, it becomes more difficult to monitor the system and identify issues.
- Maintenance: With more servers, there is more hardware and software that needs to be maintained and updated, which increases the workload and complexity of managing the infrastructure.

Container Orchestration

Use container orchestration to **automate and manage** tasks such as:

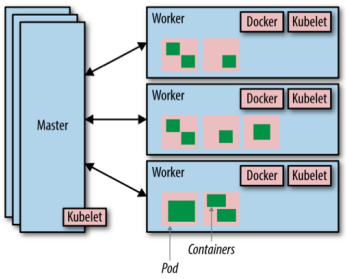
- Deployment
- Configuration
- Scheduling
- Resource allocation
- Container availability
- Load balancing and traffic routing
- Scaling or removing containers based on balancing workloads across your infrastructure
- Monitoring container health
- Keeping interactions between containers secure

Docker Swarm and Kubernetes (K8s) are both container orchestration tools that allow for the management and deployment of containerized applications **at scale**





Kubernetes architecture



Cluster: A control plane and one or more compute machines, or worker nodes.

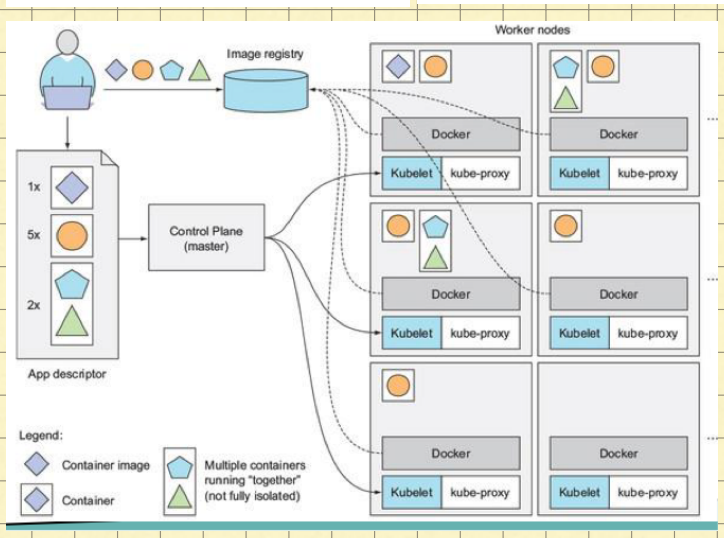
- **Control plane (master)**: The collection of processes that control k8s nodes. This is where all task assignments originate.
- **Worker nodes**: Worker nodes within the k8s cluster are used to run containerized applications

Worker node

- **Pod**: smallest deployable unit in k8s.
  - A pod hosts one or more containers
  - A pod provides shared storage and networking for its containers
- **Kubelet**: a system service (daemon) that runs on a worker node
  - Managing and monitoring the state of the pods and containers on that node.
  - Communicate with the master to receive instructions and report status updates.

Worker node

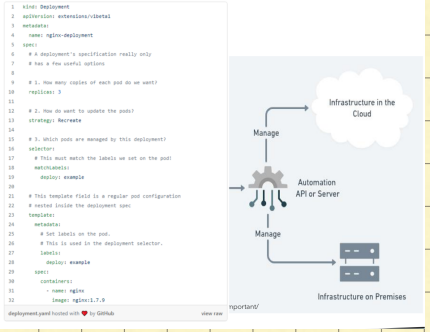
- **Container runtime** (Docker): runs the containers on worker nodes
  - Pulling container images from registry
  - Starting and stopping containers
  - Managing container resources
- **Kube-proxy**: network proxy on workers
  - Routing traffic to the correct pods
  - Load balancing



Infrastructure as code

INFRASTRUCTURE AS CODE

- Infrastructure as code (IaC) is DevOps practice that involves defining and managing infrastructure resources such as servers, networks, storage using code instead of manual processes.
- IaC code is stored in version control systems such as Git, and then executed using automation tools such as Ansible or Terraform.
- Teams can define and maintain their infrastructure as code, just like software applications, and apply best practices such as version control, testing, and CI/CD.



Cloud-native Open Standards

- A set of open-source standards and best practices that enable the development and deployment of cloud-native applications in a scalable, efficient, and portable way.
- These standards cover various aspects of cloud-native development, including containerization, microservices, orchestration, monitoring, security, etc.
- These standards ensure interoperability between different cloud environments, reduces vendor lock-in (降低对特定供应商的依赖), and allows for greater flexibility in application development and deployment.
- K8s: a de facto open standard for container orchestration
- Suppose that our application is first deployed to a K8s cluster running on Amazon Web Services (AWS) cloud platform.
- Later, we decide to switch to a different cloud platform, such as Microsoft Azure
- Since K8s is an open standard which is supported by both, our application is portable across different cloud platforms

Development Pipeline

- A software deployment pipeline is a set of automated processes that an application goes through from the development to final production

- A deployment pipeline includes various stages such as building, testing, and deploying, and different environments like staging, testing, and production.

- Ensures that the application is delivered reliably, quickly, and with high quality.
- Enables the team to deliver software changes more frequently and with greater confidence. Minimizes the risk of human error.
- Provides visibility into the progress of the application
- Enables faster feedback to developers in case of any issues or failures.

Common components of a deployment pipeline

- Version control
- Artifact repositories
- Build automation tools
- Automated testing tools
- Continuous integration (CI) servers
- Deployment tools